المذكرة الاحترافية

Software Engineering for Machine Learning Engineers

الفصل الثالث: Design Patterns

حلول جاهزة ومجربة لمشاكل تصميم متكررة، مبنية فوق مبادئ SOLID

Factory • Singleton • Strategy • Observer • Dependency Injection

جدول المحتويات

جدول المحتويات 2
مقدمة الفصل الثالث: من مبادئ SOLID إلى حلول جاهزة 5
٣،١ Factory Pattern 6
١. ليه نمط Factory مهم؟ 6
٢. المشكلة اللي بيحلها Factory 6
٣. إزاي كانت الشركات بتتعامل قبل تبني Factory 6
٤. ليه الطريقة القديمة مش مناسبة 6
٥. الفكرة الأساسية وراء Factory 6
٦. المفاهيم الأساسية 6
٧. المكونات 6
٨. إزاي المبدأ ده بيشتغل داخليًا 6
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 6
١٠. فين موقعه داخل مشروع ML حقيقي 7
١١. امتى تستخدم Factory 7
١٢. امتى Factory يكون زيادة عن اللزوم 7
١٣. أفضل الممارسات 7
١٤. أشهر الأخطاء الشائعة 7
١٥. اعتبارات الأداء 7
١٦. الاعتبارات الأمنية 7
١٧. اعتبارات قابلية التوسع 7
١٨. مقارنة مع نهج بديل 8
١٩. مثال عملي صغير 8
٢٠. مثال من مشروع Production حقيقي 8
٢١. ملخص الموضوع 8
٣،٢ Singleton Pattern 9
١. ليه نمط Singleton مهم؟ 9
٢. المشكلة اللي بيحلها Singleton 9
٣. إزاي كانت الشركات بتتعامل قبل تبني Singleton 9
٤. ليه الطريقة القديمة مش مناسبة 9
٥. الفكرة الأساسية وراء Singleton 9
٦. المفاهيم الأساسية 9
٧. المكونات 9
٨. إزاي المبدأ ده بيشتغل داخليًا 9
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 9
١٠. فين موقعه داخل مشروع ML حقيقي 10
١١. امتى تستخدم Singleton 10
١٢. امتى Singleton يكون خطر 10
١٣. أفضل الممارسات 10
١٤. أشهر الأخطاء الشائعة 10
١٥. اعتبارات الأداء 10

١٦. الاعتبارات الأمنية 10
١٧. اعتبارات قابلية التوسع 10
١٨. مقارنة مع نهج بديل 11
١٩. مثال عملي صغير 11
٢٠. مثال من مشروع Production حقيقي 11
٢١. ملخص الموضوع 11
٣،٣ Strategy Pattern 12
١. ليه نمط Strategy مهم؟ 12
٢. المشكلة اللي بيحلها Strategy 12
٣. إزاي كانت الشركات بتتعامل قبل تبني Strategy 12
٤. ليه الطريقة القديمة مش مناسبة 12
٥. الفكرة الأساسية وراء Strategy 12
٦. المفاهيم الأساسية 12
٧. المكونات 12
٨. إزاي المبدأ ده بيشتغل داخليًا 12
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 12
١٠. فين موقعه داخل مشروع ML حقيقي 13
١١. امتى تستخدم Strategy 13
١٢. امتى Strategy يكون زيادة عن اللزوم 13
١٣. أفضل الممارسات 13
١٤. أشهر الأخطاء الشائعة 13
١٥. اعتبارات الأداء 13
١٦. الاعتبارات الأمنية 13
١٧. اعتبارات قابلية التوسع 13
١٨. مقارنة مع نهج بديل 14
١٩. مثال عملي صغير 14
٢٠. مثال من مشروع Production حقيقي 14
٢١. ملخص الموضوع 14
٣،٤ Observer Pattern 15
١. ليه نمط Observer مهم؟ 15
٢. المشكلة اللي بيحلها Observer 15
٣. إزاي كانت الشركات بتتعامل قبل تبني Observer 15
٤. ليه الطريقة القديمة مش مناسبة 15
٥. الفكرة الأساسية وراء Observer 15
٦. المفاهيم الأساسية 15
٧. المكونات 15
٨. إزاي المبدأ ده بيشتغل داخليًا 15
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 15
١٠. فين موقعه داخل مشروع ML حقيقي 16
١١. امتى تستخدم Observer 16
١٢. امتى Observer يكون زيادة عن اللزوم 16

١٣. أفضل الممارسات 16
١٤. أشهر الأخطاء الشائعة 16
١٥. اعتبارات الأداء 16
١٦. الاعتبارات الأمنية 16
١٧. اعتبارات قابلية التوسع 16
١٨. مقارنة مع نهج بديل 17
١٩. مثال عملي صغير 17
٢٠. مثال من مشروع Production حقيقي 17
٢١. ملخص الموضوع 17
٣,٥ Dependency Injection Pattern 18
١. ليه نمط Dependency Injection مهم؟ 18
٢. المشكلة اللي بيحلها Dependency Injection 18
٣. إزاي كانت الشركات بتتعامل قبل تبني Dependency Injection 18
٤. ليه الطريقة القديمة مش مناسبة 18
٥. الفكرة الأساسية وراء Dependency Injection 18
٦. المفاهيم الأساسية 18
٧. المكونات 18
٨. إزاي المبدأ ده بيشتغل داخليًا 18
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 19
١٠. فين موقعه داخل مشروع ML حقيقي 19
١١. امتى تستخدم Dependency Injection 19
١٢. امتى Dependency Injection يكون زيادة عن اللزوم 19
١٣. أفضل الممارسات 19
١٤. أشهر الأخطاء الشائعة 19
١٥. اعتبارات الأداء 19
١٦. الاعتبارات الأمنية 19
١٧. اعتبارات قابلية التوسع 20
١٨. مقارنة مع نهج بديل 20
١٩. مثال عملي صغير 20
٢٠. مثال من مشروع Production حقيقي 20
٢١. ملخص الموضوع 20
مشروع عملي على الفصل: بناء Mini ML Pipeline بأنماط تصميم متكاملة 21
خطوات التنفيذ 21
تحديات شائعة عند التطبيق 21
إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي 21
ملخص الفصل الثالث وربطه بالفصل القادم 21

مقدمة الفصل الثالث: من مبادئ SOLID إلى حلول جاهزة

في الفصل اللي فات اتعلمنا خمس مبادئ (SOLID) بتحكم إزاي تصمم كلاسات وعلاقاتها ببعض بشكل مرن وقابل للتوسع. الفصل ده هياخدك خطوة عملية لقدام: Design Patterns، وهي حلول جاهزة ومجربة لمشاكل تصميم متكررة، كل واحد منها في الحقيقة تطبيق مباشر لمبدأ أو أكتر من مبادئ SOLID اللي اتعلمناها.

الفرق بين مبدأ SOLID والنمط التصميمي (Design Pattern) إن المبدأ فكرة عامة (زي مفتوح للتوسع مغلق للتعديل)، بينما النمط حل محدد وجاهز للتطبيق (زي نمط Strategy اللي بيطبق فكرة OCP دي بشكل ملموس وقابل لإعادة الاستخدام في مواقف متكررة).

هندرس في الفصل ده خمس أنماط أساسية: Factory (لإنشاء الكائنات)، Singleton (لضمان وجود نسخة واحدة بس من كلاس معين)، Strategy (لتبديل السلوك وقت التشغيل)، Observer (لإشعار عدة أطراف بتغيير معين)، وDependency Injection (لحقن الاعتماديات بدل إنشائها داخليًا).

ليه بندرس الأنماط دي دلوقتي؟ لأن الفصل الجاي (Clean Architecture) هيستخدم بعض الأنماط دي - خصوصًا Dependency Injection - كأداة أساسية لتطبيق قاعدة الاعتماد (Dependency Rule) بين طبقات النظام المختلفة. وهتلاقي كمان إن أنماط زي Factory وStrategy بتتكرر باستمرار في تصميم أي Pipeline حقيقي للـ Machine Learning.

٣,١ Factory Pattern

نمط Factory هو حل لمشكلة إنشاء الكائنات (Object Creation) لما تكون منطق اختيار النوع المطلوب إنشاؤه معقد أو متوقع يتغير مستقبلًا. بدل ما تنشئ الكائن مباشرة، بتفوض عملية الإنشاء لدالة أو كلاس متخصص اسمه Factory.

١. ليه نمط Factory مهم؟

لما تحتاج تنشئ كائن من نوع معين، وفيه أكتر من نوع ممكن (زي أنواع نماذج مختلفة، أو أنواع مصادر بيانات مختلفة)، منطق اختيار النوع الصح ده لو اتكرر في كل مكان بيستخدم فيه الكائن، أي تغيير في منطق الاختيار هيحتاج تعديل في أماكن كتيرة.

Factory بيجمع منطق الإنشاء في مكان واحد، وده بيسهل التعديل والصيانة، وبيطبق مبدأ OCP اللي اتعلمناه في الفصل اللي فات: إضافة نوع جديد بتتم بتعديل الـ Factory بس، من غير التأثير على كل الأماكن اللي بتستخدم الكائنات دي.

٢. المشكلة اللي بيحلها Factory

المشكلة هي تكرار منطق if/elif لاختيار وإنشاء نوع الكائن المطلوب في كذا مكان مختلف من الكود. لو عندك منطق زي لو النوع linear أنشئ LinearRegression، لو tree أنشئ DecisionTree في عشر أماكن مختلفة، أي نوع جديد هيحتاج تعديل في العشر أماكن دول كلهم.

٣. إزاي كانت الشركات بتتعامل قبل تبني Factory

كان شائع إنشاء الكائنات مباشرة في كل مكان محتاج ليها، باستخدام شروط if/elif متكررة لتحديد النوع المطلوب حسب المدخلات، من غير مركزية لمنطق الإنشاء ده.

٤. ليه الطريقة القديمة مش مناسبة

التكرار ده بيخالف مبدأ DRY (Don't Repeat Yourself)، وبيزود احتمال الأخطاء لو نسيت تحدث مكان واحد من الأماكن المتكررة لما تضيف نوع جديد أو تعدل منطق الاختيار.

٥. الفكرة الأساسية وراء Factory

بتعرف دالة أو كلاس واحد (الـ Factory) مسؤول بس عن استقبال المعطيات اللازمة وإرجاع الكائن المناسب من النوع الصحيح. أي كود محتاج ينشئ كائن، بينادي على الـ Factory بدل ما ينشئ الكائن مباشرة بنفسه.

٦. المفاهيم الأساسية

- Simple Factory: دالة واحدة بتستقبل نوع أو معطيات وبترجع الكائن المناسب.
- Factory Method: كل كلاس فرعي بيحدد إزاي ينشئ الكائن الخاص بيه، عن طريق Method مخصصة للإنشاء.
- Abstract Factory: مجموعة Factories مترابطة بتنشئ عائلة كاملة من الكائنات المتوافقة مع بعض.

٧. المكونات

- الواجهة أو الكلاس الأساسي المشترك بين كل الأنواع اللي الـ Factory ممكن يرجعها.
- دالة أو كلاس الـ Factory نفسه اللي بيحدد النوع المطلوب إنشاؤه ويرجعه.
- الكود المستخدم (Client Code) اللي بيطلب الكائن من الـ Factory من غير معرفة تفاصيل الإنشاء.

٨. إزاي المبدأ ده بيشتغل داخليًا

الـ Factory بيغلف منطق اتخاذ القرار (أي نوع أنشئ) في مكان واحد، والكود المستخدم بيتعامل مع الواجهة المشتركة للكائن الناتج بس، من غير ما يهتم بتفاصيل إزاي اتحدد النوع أو اتنشأ الكائن.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات الـ ML الكبيرة، بيتم استخدام Factory لإنشاء النموذج المناسب بناءً على إعدادات التجربة (Experiment Configuration)، فمثلًا ملف Configuration بيحدد نوع النموذج المطلوب كنص (زي xgboost أو neural_network)، والـ Factory بيحول النص ده لكائن النموذج الفعلي المطلوب.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في نقطة إنشاء النموذج بناءً على إعدادات التجربة، وفي إنشاء معالج البيانات المناسب حسب نوع البيانات، وفي إنشاء الاتصال بمصدر البيانات المناسب حسب البيئة (تطوير أو إنتاج).

١١. امتى تستخدم Factory

- لما منطق إنشاء الكائن معقد أو بيعتمد على شروط متعددة.
- لما نفس منطق الإنشاء متوقع يتكرر في أكتر من مكان في الكود.
- لما عدد الأنواع الممكن إنشاؤها متوقع يزيد مستقبلًا.

١٢. امتى Factory يكون زيادة عن اللزوم

- لما إنشاء الكائن بسيط جدًا (Constructor واحد بس بمعطيات مباشرة) ومفيش منطق قرار حقيقي وراه.
- لما فيه نوع واحد بس متوقع طول الوقت.

١٣. أفضل الممارسات

Best Practices

- استخدم Factory لما منطق الإنشاء فيه قرار حقيقي بناءً على معطيات متغيرة.
- خلي الـ Factory يرجع دايمًا نفس الواجهة المشتركة (Abstract Type) بغض النظر عن النوع الفعلي الناتج.
- اربط الـ Factory بنمط OCP، بحيث إضافة نوع جديد تتم بإضافة حالة جديدة في الـ Factory من غير التأثير على الكود المستخدم.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استخدام Factory لحالات بسيطة جدًا من غير أي منطق قرار حقيقي، وده بيزود التعقيد من غير فايدة.
- الـ Factory بيرجع أنواع مختلفة من غير واجهة مشتركة، وده بيصعب التعامل مع الناتج من الكود المستخدم.
- تكرار منطق الاختيار برا الـ Factory بدل تركيزه في مكان واحد بس.

١٥. اعتبارات الأداء

استخدام Factory بيضيف طبقة استدعاء إضافية بسيطة (Function Call) مقارنة بالإنشاء المباشر، لكن التكلفة دي مهملة عمليًا مقارنة بفايدة مركزية منطق الإنشاء.

١٦. الاعتبارات الأمنية

مركزية منطق الإنشاء في الـ Factory بتسمح بفرض تحقق موحد من صحة المدخلات (Input Validation) قبل إنشاء أي كائن، بدل الاعتماد على تحقق متكرر وغير متسق في كل مكان بيتم فيه الإنشاء.

١٧. اعتبارات قابلية التوسع

Factory بيسهل إضافة أنواع جديدة للنظام من غير التأثير على الكود الموجود، وده بيدعم قابلية التوسع في أنظمة بتحتاج تدعم أنواع متعددة من النماذج أو مصادر البيانات بشكل متزايد مع الوقت.

١٨. مقارنة مع نهج بديل

البديل هو الإنشاء المباشر للكائنات في كل مكان محتاج ليها. الأسلوب ده أبسط لحالة واحدة بسيطة، لكنه بيفشل بسرعة مع تعدد الأنواع وتكرار منطق الاختيار. الشركات الكبرى بتستخدم Factory بمجرد ما منطق الإنشاء يبقى معقد بما يكفي أو متوقع يتكرر.

١٩. مثال عملي صغير

```
# Before - creation logic repeated wherever a model is needed
if model_type == 'linear':
    model = LinearRegression()
elif model_type == 'tree':
    model = DecisionTreeRegressor()

# After - creation logic centralized in a factory
class ModelFactory:
    _registry = {
        'linear': LinearRegression,
        'tree': DecisionTreeRegressor,
    }

    @classmethod
    def create(cls, model_type: str):
        if model_type not in cls._registry:
            raise ValueError(f'Unknown model type: {model_type}')
        return cls._registry[model_type]()

model = ModelFactory.create(model_type)
```

٢٠. مثال من مشروع Production حقيقي

في منصة ML كبيرة بتدعم عشرات أنواع النماذج، ملف Configuration بيحدد نوع النموذج المطلوب لكل تجربة كنص بسيط، وModelFactory واحد مسؤول عن تحويل النص ده لكائن النموذج الفعلي. لما فريق النمذجة يضيف خوارزمية جديدة، بيسجلوها في الـ Factory بس، من غير أي تعديل في كود الـ Pipeline الرئيسي.

٢١. ملخص الموضوع

خلاصة Factory Pattern

- Factory بيجمع منطق إنشاء الكائنات في مكان واحد بدل تكراره.
- بيطبق مبدأ OCP: إضافة نوع جديد تتم في الـ Factory بس.
- استخدمه لما منطق الإنشاء معقد أو متكرر، وتجنبه للحالات البسيطة.

Singleton Pattern ٣,٢

نمط Singleton بيضمن إن كلاس معين ليه نسخة واحدة بس (Single Instance) في كل البرنامج، ويوفر نقطة وصول عامة (Global Access Point) للنسخة دي.

١. ليه نمط Singleton مهم؟

بعض الموارد في النظام لازم تكون نسخة واحدة بس بطبيعتها: اتصال بقاعدة بيانات، إعدادات النظام (Configuration)، أو Connection Pool. لو اتعملت أكتر من نسخة من الموارد دي بالغلط، ممكن يحصل تضارب في الحالة (State) أو استهلاك غير ضروري للموارد.

في مشاريع ML، مثال شائع هو كلاس Configuration اللي بيحمل إعدادات التجربة، لازم يكون نسخة واحدة بس متسقة عبر كل أجزاء الـ Pipeline.

٢. المشكلة اللي بيحلها Singleton

المشكلة هي إمكانية إنشاء أكتر من نسخة من كلاس المفروض يكون فريد بطبيعته، وده ممكن يؤدي لتضارب في الحالة (زي نسختين من الإعدادات بقيم مختلفة في نفس الوقت)، أو استهلاك مضاعف للموارد (زي فتح أكتر من اتصال بقاعدة بيانات من غير داعي).

٣. إزاي كانت الشركات بتتعامل قبل تبني Singleton

كان شائع الاعتماد على متغيرات عامة (Global Variables) مباشرة للموارد المشتركة دي، من غير ضمان حقيقي إنها بتتنشأ مرة واحدة بس أو إنها بتتحدث بشكل متسق عبر النظام.

٤. ليه الطريقة القديمة مش مناسبة

المتغيرات العامة المباشرة مش بتوفر أي ضمان أو تحكم في عملية الإنشاء، وأي جزء من الكود ممكن يعيد تعريفها أو يغيرها بشكل غير متوقع، وده بيصعب تتبع مصدر المشاكل لما يحصل تضارب في الحالة.

٥. الفكرة الأساسية وراء Singleton

الكلاس نفسه بيتحكم في عملية إنشائه، بحيث أي محاولة لإنشاء نسخة جديدة بترجع نفس النسخة الموجودة بالفعل لو كانت موجودة، أو تنشئ نسخة جديدة وتحفظها لو دي أول مرة.

٦. المفاهيم الأساسية

- Single Instance Guarantee: ضمان إن نسخة واحدة بس من الكلاس موجودة طول عمر البرنامج.
- Global Access Point: نقطة وصول موحدة للنسخة دي من أي مكان في الكود.
- Lazy Initialization: إنشاء النسخة أول مرة بس لما تتطلب فعليًا، مش عند بدء تشغيل البرنامج.

٧. المكونات

- متغير خاص (Private) بيحمل النسخة الوحيدة، عادة على مستوى الكلاس نفسه.
- Method أو خاصية عامة (زي get_instance) بترجع النسخة الموجودة أو تنشئها لو أول مرة.
- منع الإنشاء المباشر للكلاس من برا، عشان تضمن إن الوصول بيتم بس عن طريق Method الوصول الموحدة.

٨. إزاي المبدأ ده بيشتغل داخليًا

أول مرة حد يطلب النسخة، الكلاس بينشئها ويحفظها في متغير ثابت على مستوى الكلاس. أي طلب تاني بعد كده بيرجع نفس النسخة المحفوظة دي بدل ما ينشئ نسخة جديدة، وده بيضمن إن كل أجزاء البرنامج بتتعامل مع نفس الحالة المشتركة.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في أنظمة ML الكبيرة، كلاس إدارة الاتصال بموارد مشتركة زي Feature Store أو Model Registry غالبًا بيتصمم كـ Singleton، لضمان إن كل أجزاء النظام بتستخدم نفس الاتصال المُدار بكفاءة، بدل فتح اتصالات متعددة غير ضرورية.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في كلاس إدارة إعدادات التجربة (Experiment Configuration)، وكلاس إدارة الاتصال بمصادر البيانات المشتركة، وأحيانًا في كلاس تسجيل الأحداث (Logger) المشترك عبر النظام كله.

١١. امتى تستخدم Singleton

- لما المورد بطبيعته لازم يكون نسخة واحدة بس (اتصال، إعدادات، Connection Pool).
- لما إنشاء نسخ متعددة هيسبب تضارب حقيقي في الحالة أو استهلاك زايد للموارد.

١٢. امتى Singleton يكون خطر

- لما بتستخدمه لتخزين حالة مشتركة (Shared Mutable State) بشكل مفرط، وده بيصعب الاختبار لأن الحالة بتفضل موجودة بين الاختبارات المختلفة.
- في بيئات متوازية (Concurrent) بدون التعامل الصحيح مع التزامن (Thread Safety)، ممكن ينشئ أكتر من نسخة بالغلط.

١٣. أفضل الممارسات

Best Practices

- استخدم Singleton بس للموارد اللي فعلًا لازم تكون فريدة بطبيعتها.
- لو النظام بيشتغل في بيئة متوازية (Multi-threaded)، تأكد من تطبيق Singleton بشكل آمن للتزامن (Thread-Safe).
- فضل حقن الاعتماد (Dependency Injection) على Singleton لما تحتاج مرونة أكبر في الاختبار.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استخدام Singleton كبديل كسول للمتغيرات العامة، بدون تفكير حقيقي في الحاجة الفعلية للنسخة الواحدة.
- تخزين حالة قابلة للتغيير (Mutable State) بشكل مفرط في Singleton، وده بيصعب الاختبار بسبب تسرب الحالة بين اختبارات مختلفة.
- عدم التعامل مع التزامن (Thread Safety) في بيئات متوازية، وده ممكن يؤدي لإنشاء أكتر من نسخة بالخطأ.

١٥. اعتبارات الأداء

Singleton بيوفر أداء أفضل في حالات معينة، زي تجنب إنشاء اتصالات متعددة مكلفة (زي اتصالات قواعد البيانات)، لكن لازم الانتباه لتكلفة التزامن (Locking) في البيئات المتوازية لضمان إنشاء نسخة واحدة بس بأمان.

١٦. الاعتبارات الأمنية

الوصول المشترك لنسخة واحدة بيتطلب انتباه خاص لصلاحيات الوصول، خصوصًا لو الـ Singleton بيحمل بيانات حساسة زي إعدادات اتصال بقاعدة بيانات، لأن أي جزء من الكود عنده وصول للنسخة المشتركة دي.

١٧. اعتبارات قابلية التوسع

Singleton ممكن يكون عائق أمام التوسع الأفقي (Horizontal Scaling) في بعض الحالات، خصوصًا لو النظام بيتوسع لأكتر من عملية (Process) أو خادم (Server)، لأن كل عملية هتكون ليها نسخة Singleton منفصلة، وده ممكن يخالف افتراض النسخة الواحدة على مستوى النظام كله.

١٨. مقارنة مع نهج بديل

البديل هو حقن الاعتماد (Dependency Injection)، حيث بدل ما الكلاس يضمن نسخته الوحيدة بنفسه، النسخة الوحيدة بتتنشأ مرة واحدة في نقطة مركزية (زي عند بدء تشغيل التطبيق) وبتتمرر لكل الكلاسات المحتاجاها. الأسلوب ده أسهل في الاختبار من Singleton التقليدي، والشركات الكبرى بتميل ليه أكتر في الأنظمة الحديثة.

١٩. مثال عملي صغير

```
class ExperimentConfig:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance

    def __init__(self, settings_path: str = None):
        if self._initialized:
            return
        self.settings_path = settings_path
        self._initialized = True

# Both calls return the exact same instance
config_a = ExperimentConfig('config.yaml')
config_b = ExperimentConfig()
assert config_a is config_b
```

٢٠. مثال من مشروع Production حقيقي

في نظام ML بيشتغل على منصة سحابية، كلاس FeatureStoreConnection بيتصمم كـ Singleton عشان يضمن إن كل أجزاء الـ Pipeline (تحميل بيانات، تدريب، تقييم) بتستخدم نفس الاتصال المُدار بكفاءة بدل فتح اتصالات منفصلة مكلفة لكل جزء.

٢١. ملخص الموضوع

خلاصة Singleton Pattern

- Singleton بيضمن نسخة واحدة بس من كلاس معين مع نقطة وصول عامة ليها.
- استخدمه بس للموارد اللي فعلًا لازم تكون فريدة (اتصال، إعدادات مشتركة).
- انتبه لمشاكل الاختبار والتزامن، وفضل Dependency Injection لو محتاج مرونة أكبر.

٣,٣ Strategy Pattern

نمط Strategy بيسمح باختيار وتبديل خوارزمية أو سلوك معين وقت التشغيل (Runtime)، عن طريق تعريف مجموعة من الخوارزميات القابلة للتبديل خلف واجهة موحدة. ده في الحقيقة تطبيق عملي مباشر لمبدأ Open/Closed اللي اتعلمناه في الفصل السابق.

١. ليه نمط Strategy مهم؟

كتير من الأنظمة محتاجة تدعم أكتر من طريقة لأداء نفس المهمة، زي أكتر من خوارزمية Normalization، أو أكتر من طريقة لحساب المقاييس. Strategy بيسمح بتبديل الخوارزمية المستخدمة من غير تغيير الكود اللي بيستخدمها.

في مشاريع ML، ده بيظهر باستمرار: نفس الـ Pipeline ممكن يحتاج يشتغل بخوارزميات تدريب مختلفة، أو طرق معالجة بيانات مختلفة، حسب التجربة المطلوبة.

٢. المشكلة اللي بيحلها Strategy

المشكلة هي الكود اللي بيحتوي على شروط متعددة لاختيار الخوارزمية المطلوبة (زي شروط if/elif طويلة لاختيار طريقة الـ Normalization)، وأي إضافة لخوارزمية جديدة بتتطلب تعديل نفس الكود المركزي ده، وده تحديدًا نفس المشكلة اللي واجهناها في مبدأ OCP.

٣. إزاي كانت الشركات بتتعامل قبل تبني Strategy

كان شائع كتابة دالة واحدة بتحتوي على كل الخوارزميات الممكنة جوه شروط if/elif، وكل خوارزمية جديدة بتتضاف كحالة جديدة في نفس الدالة.

٤. ليه الطريقة القديمة مش مناسبة

الدالة الواحدة اللي بتحتوي كل الخوارزميات بتكبر وتتعقد مع كل إضافة جديدة، وأي تعديل في خوارزمية معينة بيحمل خطر التأثير على الخوارزميات التانية اللي في نفس الدالة.

٥. الفكرة الأساسية وراء Strategy

كل خوارزمية بتتنفذ كتطبيق منفصل لواجهة موحدة، والكود المستخدم بيستقبل الخوارزمية المطلوبة (Strategy) كمعطى، وينفذها عن طريق الواجهة الموحدة دي من غير ما يهتم بتفاصيل كل خوارزمية على حدة.

٦. المفاهيم الأساسية

- Interchangeable Algorithms: خوارزميات متعددة قابلة للتبديل خلف نفس الواجهة.
- Runtime Selection: اختيار الخوارزمية المطلوبة وقت التشغيل، مش وقت كتابة الكود.
- Context Class: الكلاس اللي بيستخدم الـ Strategy، وبيحتفظ بمرجع للاستراتيجية الحالية المختارة.

٧. المكونات

- واجهة موحدة (Strategy Interface) بتحدد الشكل العام لكل الخوارزميات.
- تطبيقات محددة (Concrete Strategies) لكل خوارزمية فعلية.
- كلاس السياق (Context) اللي بيستخدم الاستراتيجية المختارة من غير معرفة تفاصيلها الداخلية.

٨. إزاي المبدأ ده بيشتغل داخليًا

كلاس السياق (Context) بيحتفظ بمرجع (Reference) لكائن الاستراتيجية الحالية، ولما يحتاج ينفذ العملية، بينادي على Method موحدة في الاستراتيجية دي. أي استراتيجية جديدة بتتنفذ نفس الواجهة، فكلاس السياق مش محتاج أي تعديل عشان يدعمها.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في مكتبات ML الكبرى، نمط Strategy بيظهر بوضوح في اختيار دالة الخسارة (Loss Function) أو خوارزمية التحسين (Optimizer)، حيث الكود الأساسي للتدريب بيتعامل مع واجهة موحدة، والمستخدم بيقدر يمرر أي دالة خسارة أو محسن متوافق مع الواجهة دي.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في اختيار طريقة الـ Normalization أو الـ Feature Scaling، اختيار خوارزمية التدريب، اختيار طريقة تقييم النموذج (Evaluation Metric)، واختيار استراتيجية أخذ العينات (Sampling Strategy) في التعامل مع البيانات غير المتوازنة.

١١. امتى تستخدم Strategy

- لما عندك أكتر من طريقة لأداء نفس المهمة، ومتوقع الحاجة للتبديل بينهم وقت التشغيل.
- لما عدد الطرق البديلة متوقع يزيد مستقبلًا.

١٢. امتى Strategy يكون زيادة عن اللزوم

- لما فيه طريقة واحدة بس ثابتة ومش متوقع تتغير أو تتعدد مستقبلًا.
- لما الفرق بين الطرق البديلة بسيط جدًا (زي قيمة واحدة مختلفة) ومش محتاج طبقة تجريد كاملة.

١٣. أفضل الممارسات

Best Practices

- عرف واجهة واضحة ومحددة لكل الاستراتيجيات البديلة.
- خلي كلاس السياق (Context) يتعامل مع الواجهة المجردة بس، مش الاستراتيجيات المحددة مباشرة.
- استخدم Strategy لما فيه حاجة حقيقية لتبديل السلوك وقت التشغيل، مش كنمط افتراضي لكل حالة.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- الاستمرار في استخدام شروط if/elif لاختيار السلوك بدل تطبيق Strategy، رغم تعدد الحالات وتكرارها.
- خلط منطق اختيار الاستراتيجية مع منطق تنفيذها في نفس المكان.
- بناء طبقة Strategy معقدة لحالة بديلة واحدة بس مش متوقع لها تعدد حقيقي.

١٥. اعتبارات الأداء

استدعاء الاستراتيجية عن طريق الواجهة الموحدة بيضيف تكلفة بسيطة جدًا مقارنة بالاستدعاء المباشر، وهي مهملة عمليًا في أغلب حالات الاستخدام في مشاريع ML.

١٦. الاعتبارات الأمنية

لازم الانتباه لو الاستراتيجية بتتحدد من مدخلات خارجية (زي اسم الاستراتيجية جاي من طلب مستخدم)، يجب التحقق من إن الاسم ده يقابل استراتيجية مسجلة فعلًا، تجنبًا لتنفيذ كود غير متوقع.

١٧. اعتبارات قابلية التوسع

Strategy بيسمح بإضافة خوارزميات جديدة من غير التأثير على كلاس السياق أو باقي الاستراتيجيات الموجودة، وده بيدعم التوسع المستمر في الأنظمة اللي بتحتاج تجربة خوارزميات جديدة باستمرار زي أنظمة الـ ML التجريبية.

١٨. مقارنة مع نهج بديل

البديل هو تمرير دالة مباشرة (Function as Parameter) بدل كائن استراتيجية كامل، وهو أسلوب مقبول في اللغات اللي بتدعم الدوال ككائنات من الدرجة الأولى (First-Class Functions) زي Python، ويحقق نفس الفايدة بشكل أبسط لو الاستراتيجية مش محتاجة تحتفظ بحالة داخلية معقدة.

١٩. مثال عملي صغير

```
from abc import ABC, abstractmethod

class ScalingStrategy(ABC):
    @abstractmethod
    def scale(self, data):
        ...

class MinMaxScaling(ScalingStrategy):
    def scale(self, data):
        return (data - data.min()) / (data.max() - data.min())

class StandardScaling(ScalingStrategy):
    def scale(self, data):
        return (data - data.mean()) / data.std()

class FeaturePreprocessor:
    def __init__(self, strategy: ScalingStrategy):
        self.strategy = strategy

    def process(self, data):
        return self.strategy.scale(data)

preprocessor = FeaturePreprocessor(StandardScaling())
scaled_data = preprocessor.process(raw_data)
```

٢٠. مثال من مشروع Production حقيقي

في نظام تجريب ML حقيقي، فريق البحث بيقدر يجرب أكتر من استراتيجية لأخذ العينات (Sampling Strategy) في التعامل مع بيانات غير متوازنة (زي Random Oversampling أو SMOTE)، عن طريق تمرير استراتيجية مختلفة لنفس كلاس الـ Pipeline، من غير أي تعديل في كود الـ Pipeline نفسه.

٢١. ملخص الموضوع

خلاصة Strategy Pattern

- Strategy بيسمح بتبديل الخوارزمية أو السلوك وقت التشغيل خلف واجهة موحدة.
- هو التطبيق العملي المباشر لمبدأ OCP في مواجهة تعدد الخوارزميات.
- استخدمه لما فيه تعدد حقيقي متوقع في طرق أداء نفس المهمة.

Observer Pattern ٣,٤

نمط Observer بيعرف علاقة واحد لكذا (One-to-Many) بين كائن اسمه Subject وعدة كائنات تانية اسمها Observers، بحيث لما حالة الـ Subject تتغير، كل الـ Observers المسجلين بيتم إشعارهم أوتوماتيكيًا.

١. ليه نمط Observer مهم؟

كتير من الأنظمة محتاجة إن أجزاء متعددة تستجيب لحدث معين، من غير ما يكون فيه اعتماد مباشر (Tight Coupling) بين الجزء اللي بيولد الحدث والأجزاء اللي بتستجيب ليه. Observer بيوفر آلية لفصل الطرفين دول عن بعض.

في مشاريع ML، مثال شائع هو نظام مراقبة التدريب (Training Monitoring): لما مقياس معين (Metric) يتحدث بعد كل Epoch، عدة أجزاء محتاجة تستجيب (تسجيل في Log، تحديث لوحة متابعة Dashboard، إرسال تنبيه لو الأداء ساء).

٢. المشكلة اللي بيحلها Observer

المشكلة هي الاعتماد المباشر بين الجزء اللي بيولد الحدث والأجزاء اللي بتستجيب ليه، زي لو دالة تدريب النموذج بتنادي مباشرة على دالة تسجيل الـ Log، ودالة تحديث الـ Dashboard، ودالة إرسال التنبيه، كل ده جوه نفس دالة التدريب. أي إضافة لاستجابة جديدة بتتطلب تعديل دالة التدريب نفسها.

٣. إزاي كانت الشركات بتتعامل قبل تبني Observer

كان شائع استدعاء كل الاستجابات المطلوبة مباشرة وبالترتيب جوه نفس الدالة اللي بتولد الحدث، من غير آلية عامة للتسجيل والإشعار.

٤. ليه الطريقة القديمة مش مناسبة

الاستدعاء المباشر لكل الاستجابات بيخلق اعتماد قوي بين الجزء المولد للحدث وكل الأجزاء المستجيبة، وأي استجابة جديدة (زي إضافة نظام تنبيه جديد) بتتطلب تعديل الكود الأساسي للتدريب، وده يخالف مبدأ OCP اللي اتعلمناه في الفصل السابق.

٥. الفكرة الأساسية وراء Observer

الـ Subject بيحتفظ بقائمة من الـ Observers المسجلين، ولما حالته تتغير، بينادي على Method موحدة (زي update) في كل Observer مسجل، من غير ما يعرف تفاصيل كل Observer أو إيه اللي هيعمله بالضبط.

٦. المفاهيم الأساسية

- Subject (أو Publisher): الكائن اللي بيولد الأحداث ويحتفظ بقائمة المشتركين.
- Observer (أو Subscriber): الكائن اللي بيستجيب للحدث لما يوصله إشعار.
- Subscribe/Unsubscribe: آلية تسجيل وإلغاء تسجيل الـ Observers في الـ Subject وقت التشغيل.

٧. المكونات

- واجهة Observer موحدة، بتحدد Method الإشعار (زي update).
- كلاس الـ Subject، بيحتفظ بقائمة الـ Observers وبينفذ عملية الإشعار.
- تطبيقات محددة للـ Observer، كل واحدة بتستجيب للحدث بطريقتها الخاصة.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما حالة الـ Subject تتغير، بيلف على قائمة الـ Observers المسجلين وينادي على Method الإشعار الموحدة في كل واحد فيهم. كل Observer بيقرر بنفسه إزاي يستجيب للإشعار ده، والـ Subject مش بيهتم أو يعرف تفاصيل الاستجابة دي.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في أطر عمل (Frameworks) التدريب الكبيرة زي PyTorch Lightning أو Keras، نظام الـ Callbacks بالكامل مبني على فكرة Observer: كل Callback (تسجيل Log، حفظ Checkpoint، إيقاف مبكر Early Stopping) بيتسجل عند بداية التدريب، وبيتم إشعاره أوتوماتيكيًا عند كل حدث (نهاية Epoch، نهاية Batch) من غير أي تعديل في كود التدريب الأساسي.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر بوضوح في أنظمة الـ Callbacks أثناء التدريب، وفي أنظمة مراقبة النماذج في الإنتاج (Model Monitoring) حيث عدة أنظمة محتاجة تستجيب لتغير في مقاييس الأداء، وفي أنظمة الإشعارات المرتبطة بأحداث الـ Pipeline المختلفة.

١١. امتى تستخدم Observer

- لما عدة أجزاء من النظام محتاجة تستجيب لنفس الحدث بشكل مستقل.
- لما عدد المستجيبين للحدث متوقع يتغير أو يزيد مستقبلًا.

١٢. امتى Observer يكون زيادة عن اللزوم

- لما فيه استجابة واحدة بس بسيطة وثابتة للحدث، فاستدعاء مباشر بيكون أبسط وأوضح.
- لما ترتيب تنفيذ الاستجابات مهم بشكل صارم ومعقد، لأن Observer عادة بيفترض إن الاستجابات مستقلة عن بعضها.

١٣. أفضل الممارسات

Best Practices

- خلي كل Observer مستقل تمامًا عن باقي الـ Observers، ومتخليش استجابة واحدة تعتمد على ترتيب تنفيذ استجابة تانية.
- وفر آلية واضحة لتسجيل وإلغاء تسجيل الـ Observers وقت التشغيل.
- تعامل مع الأخطاء في كل Observer بشكل منفصل، عشان فشل Observer واحد ميوقفش باقي الـ Observers.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- خلق اعتماد بين الـ Observers على بعض أو على ترتيب تنفيذهم، وده بيخالف فكرة الاستقلالية الأساسية للنمط.
- عدم التعامل مع الأخطاء بشكل منفصل، بحيث فشل Observer واحد بيوقف باقي الإشعارات أو حتى الـ Subject نفسه.
- استخدام Observer لحالة بسيطة جدًا فيها استجابة واحدة بس، وده بيزود تعقيد غير ضروري.

١٥. اعتبارات الأداء

لو عدد الـ Observers كبير جدًا أو الاستجابات بطيئة (زي إرسال طلب شبكة)، الإشعار المتسلسل (Synchronous) لكل الـ Observers ممكن يبطئ الـ Subject نفسه. في الحالات دي، ممكن تحتاج تنفيذ الإشعارات بشكل غير متزامن (Asynchronous) لتقليل التأثير على الأداء.

١٦. الاعتبارات الأمنية

لازم التحقق من هوية أي Observer قبل تسجيله في الـ Subject، خصوصًا لو الأحداث بتحمل بيانات حساسة، لتجنب تسريب البيانات دي لأطراف غير مصرح لها بالوصول.

١٧. اعتبارات قابلية التوسع

Observer بيسمح بإضافة استجابات جديدة للأحداث من غير التأثير على الجزء المولد للحدث، وده بيدعم التوسع المستمر في الأنظمة اللي بتحتاج تضيف أنواع جديدة من المراقبة أو الإشعارات باستمرار.

١٨. مقارنة مع نهج بديل

البديل هو الاستدعاء المباشر لكل الاستجابات المطلوبة جوه نفس الدالة المولدة للحدث. الأسلوب ده أبسط لحالة واحدة أو اتنين، لكنه بيفشل بسرعة مع تعدد الاستجابات المطلوبة، والشركات الكبرى بتفضل نظام Observer أو Callbacks بمجرد ما عدد الاستجابات يزيد عن حالة بسيطة واحدة.

١٩. مثال عملي صغير

```
from abc import ABC, abstractmethod

class TrainingObserver(ABC):
    @abstractmethod
    def on_epoch_end(self, epoch, metrics):
        ...

class LoggingObserver(TrainingObserver):
    def on_epoch_end(self, epoch, metrics):
        print(f'Epoch {epoch}: {metrics}')

class EarlyStoppingObserver(TrainingObserver):
    def on_epoch_end(self, epoch, metrics):
        if metrics['val_loss'] > self.best_loss:
            self.patience_counter += 1

class Trainer:
    def __init__(self):
        self.observers = []

    def register(self, observer: TrainingObserver):
        self.observers.append(observer)

    def notify_epoch_end(self, epoch, metrics):
        for observer in self.observers:
            observer.on_epoch_end(epoch, metrics)
```

٢٠. مثال من مشروع Production حقيقي

في نظام تدريب حقيقي بيستخدم framework زي PyTorch Lightning، فريق النمذجة بيقدر يضيف Callback جديد (زي تسجيل المقاييس في نظام مراقبة خارجي مثل MLflow) عن طريق تسجيله كـ Observer، من غير أي تعديل في حلقة التدريب الأساسية (Training Loop) نفسها.

٢١. ملخص الموضوع

خلاصة Observer Pattern

- Observer بيسمح بإشعار عدة أطراف بتغيير معين من غير اعتماد مباشر بينهم.
- الـ Subject بيحتفظ بقائمة Observers وينادي عليهم عند حدوث الحدث.
- استخدمه لما عدة أجزاء مستقلة محتاجة تستجيب لنفس الحدث.

٣,٥ Dependency Injection Pattern

Dependency Injection هو النمط العملي اللي بيطبق مبدأ Dependency Inversion اللي اتعلمناه في الفصل السابق. بدل ما الكلاس ينشئ الاعتماديات (Dependencies) بتاعته بنفسه، بيتم تمريرها (حقنها) له من برا.

١. ليه نمط Dependency Injection مهم؟

لو كلاس بينشئ الاعتماديات بتاعته بنفسه جوه الـ Constructor، فهو بيصبح مرتبط بشكل صارم (Tightly Coupled) بتفاصيل تقنية محددة، وده بيصعب استبدال الاعتماد ده أو اختباره بمعزل عن التفاصيل الحقيقية.

في مشاريع ML، ده بيسمح مثلًا لكلاس ModelTrainer يشتغل مع أي مصدر بيانات (ملف، قاعدة بيانات، API) من غير ما يعرف تفاصيل المصدر ده، طالما بيلتزم بالواجهة المتوقعة.

٢. المشكلة اللي بيحلها Dependency Injection

المشكلة هي الكلاس اللي بينشئ الاعتماديات بتاعته داخليًا (زي self.db = DatabaseConnection() جوه الـ Constructor)، وده بيخلي اختبار الكلاس ده صعب لأنك مضطر تتعامل مع اتصال قاعدة بيانات حقيقي، وأي تغيير في نوع الاعتماد بيتطلب تعديل الكلاس نفسه.

٣. إزاي كانت الشركات بتتعامل قبل تبني Dependency Injection

كان شائع إنشاء الاعتماديات مباشرة جوه الكلاسات المحتاجة ليها، من غير أي آلية لتمريرها من برا، وده كان بيخلي الاختبار يعتمد على موارد حقيقية (قواعد بيانات، ملفات، شبكة) بدل بيانات وهمية بسيطة.

٤. ليه الطريقة القديمة مش مناسبة

الاعتماد الداخلي على تفاصيل محددة بيصعب الاختبار (لازم موارد حقيقية)، ويصعب إعادة الاستخدام (الكلاس مرتبط بتفصيل تقني واحد بس)، ويصعب الصيانة (أي تغيير في التفاصيل التقنية بيتطلب تعديل الكلاس الأساسي).

٥. الفكرة الأساسية وراء Dependency Injection

بدل ما الكلاس ينشئ اعتمادياته بنفسه، بيستقبلها كمعطيات (Parameters) عن طريق الـ Constructor أو Method مخصصة، وبيكون مسؤوليته بس استخدام الاعتماد ده من خلال واجهة متفق عليها، مش إنشاؤه.

٦. المفاهيم الأساسية

- Constructor Injection: تمرير الاعتماديات عن طريق الـ Constructor وقت إنشاء الكائن، وهو الأسلوب الأكثر شيوعًا.
- Setter Injection: تمرير الاعتماديات عن طريق Methods مخصصة بعد إنشاء الكائن.
- Dependency Injection Container: أداة أو Framework بيدير عملية إنشاء وحقن الاعتماديات أوتوماتيكيًا في تطبيقات كبيرة ومعقدة.

٧. المكونات

- الكلاس المستهلك (Consumer) اللي بيحتاج الاعتماد ويستقبله من برا.
- الواجهة المجردة (Abstraction) اللي الاعتماد بيلتزم بيها.
- نقطة التجميع (Composition Root) اللي بتنشئ الاعتماديات الفعلية وتمررها للكلاسات المحتاجة ليها، عادة في بداية تشغيل التطبيق.

٨. إزاي المبدأ ده بيشتغل داخليًا

الكلاس المستهلك بيعرف بس شكل الواجهة اللي هيتعامل معاها، مش التفاصيل الفعلية للتطبيق. الاعتماد الفعلي بيتحدد في نقطة مركزية واحدة (Composition Root)، وبيتمرر للكلاس وقت إنشائه. ده بيخلي استبدال الاعتماد (زي التبديل بين بيئة تطوير وإنتاج) بسيط جدًا، لأنه مجرد تغيير في نقطة التجميع المركزية.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في أنظمة ML الكبيرة، بيئة الاختبار (Testing) بتستخدم اعتماديات وهمية (Mock Dependencies) بدل الاتصال بموارد حقيقية زي قواعد البيانات أو الـ Feature Store، وده ممكن بس بسبب إن الكلاسات مصممة لاستقبال الاعتماديات من برا بدل إنشائها داخليًا.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في كل كلاس بيعتمد على موارد خارجية: كلاس التدريب بيستقبل مصدر البيانات كاعتماد، كلاس التقييم بيستقبل مقياس التقييم المطلوب كاعتماد، كلاس النشر بيستقبل خدمة التخزين المطلوبة كاعتماد.

١١. امتى تستخدم Dependency Injection

- لما الكلاس بيعتمد على موارد خارجية (قواعد بيانات، ملفات، خدمات شبكة) محتاج تستبدلها بسهولة للاختبار أو لتغيير البيئة.
- لما محتاج مرونة في التبديل بين تطبيقات مختلفة لنفس الاعتماد حسب السياق (تطوير مقابل إنتاج).

١٢. امتى Dependency Injection يكون زيادة عن اللزوم

- لما الاعتماد بسيط جدًا ومش متوقع يتغير أو يحتاج استبدال في الاختبار (زي قيمة ثابتة بسيطة).
- في سكربتات صغيرة جدًا ومؤقتة.

١٣. أفضل الممارسات

Best Practices

- مرر الاعتماديات عن طريق الـ Constructor عادة، لضمان إن الكائن دايمًا في حالة صالحة بمجرد إنشائه.
- خلي الكلاس المستهلك يعتمد على الواجهة المجردة، مش التطبيق الفعلي المحدد.
- ركز عملية إنشاء الاعتماديات الفعلية في نقطة واحدة (Composition Root) بدل توزيعها في أماكن متفرقة.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- إنشاء الاعتماديات داخليًا جوه الكلاس بدل حقنها من برا.
- الاعتماد على تطبيق محدد (Concrete Class) بدل الواجهة المجردة، وده بيقلل فايدة الحقن نفسها.
- تشتيت منطق إنشاء الاعتماديات في أماكن متفرقة بدل مركزته في نقطة واحدة.

١٥. اعتبارات الأداء

Dependency Injection مالوش تأثير سلبي ملحوظ على الأداء في أغلب الحالات، لأنه مجرد إعادة تنظيم لمكان إنشاء الكائنات، مش تغيير في منطق التنفيذ الفعلي.

١٦. الاعتبارات الأمنية

مركزية إنشاء الاعتماديات الحساسة (زي اتصالات قواعد البيانات) في نقطة واحدة بتسهل فرض سياسات أمان موحدة، زي التحقق من صحة بيانات الاتصال أو تشفيرها، بدل تكرار المنطق ده في كل مكان.

١٧. اعتبارات قابلية التوسع

Dependency Injection بيسهل التوسع لدعم بيئات وتقنيات جديدة، لأن استبدال الاعتماد الفعلي (زي التبديل من قاعدة بيانات لأخرى) بيتم في نقطة التجميع المركزية فقط، من غير التأثير على منطق الأعمال الأساسي.

١٨. مقارنة مع نهج بديل

البديل هو Service Locator، حيث الكلاس بيطلب الاعتماد بتاعه من مكان مركزي بنفسه، بدل ما يستقبله كمعطى. الأسلوب ده بيخفي الاعتماديات الفعلية للكلاس (مش واضحة من الـ Constructor)، وده بيصعب فهم متطلبات الكلاس بمجرد النظر لتوقيعه. الشركات الكبرى بتفضل Dependency Injection لأنه بيخلي الاعتماديات واضحة وصريحة.

١٩. مثال عملي صغير

```
from abc import ABC, abstractmethod

class MetricsSink(ABC):
    @abstractmethod
    def record(self, name, value):
        ...

class ConsoleMetricsSink(MetricsSink):
    def record(self, name, value):
        print(f'{name}: {value}')

class ModelEvaluator:
    def __init__(self, metrics_sink: MetricsSink):
        self.metrics_sink = metrics_sink

    def evaluate(self, model, data):
        accuracy = model.score(data['X'], data['y'])
        self.metrics_sink.record('accuracy', accuracy)
        return accuracy

# Composition root - the only place that decides the real implementation
evaluator = ModelEvaluator(metrics_sink=ConsoleMetricsSink())
```

٢٠. مثال من مشروع Production حقيقي

في نظام ML بيشتغل على منصة سحابية، كلاس ModelEvaluator بيستقبل MetricsSink كاعتماد محقون. في بيئة الاختبار بيتم حقن نسخة وهمية بسيطة بتحفظ القيم في الذاكرة، وفي بيئة الإنتاج بيتم حقن نسخة حقيقية بترسل المقاييس لنظام مراقبة سحابي، من غير أي تعديل في منطق التقييم نفسه.

٢١. ملخص الموضوع

خلاصة Dependency Injection

- الكلاس لازم يستقبل اعتمادياته من برا بدل ما ينشئها بنفسه.
- هو التطبيق العملي المباشر لمبدأ Dependency Inversion.
- بيسهل الاختبار والتبديل بين بيئات مختلفة من غير تعديل منطق الأعمال الأساسي.

مشروع عملي على الفصل: بناء Mini ML Pipeline بأنماط تصميم متكاملة

المشروع ده هدفه إنك تدمج الخمس أنماط (Factory، Singleton، Strategy، Observer، Dependency Injection) في تصميم واحد متكامل لـ Mini Pipeline بسيط لتدريب نموذج.

السيناريو: عايز تبني نظام تدريب يدعم أكتر من نوع نموذج، وأكتر من طريقة معالجة بيانات، مع نظام مراقبة يستجيب لكل Epoch، واعتماديات قابلة للاستبدال بين بيئة التطوير والإنتاج.

خطوات التنفيذ

- استخدم Factory لإنشاء نوع النموذج المطلوب بناءً على إعدادات التجربة (tree، linear، أو أي نوع تاني).
- استخدم Strategy لتبديل طريقة معالجة البيانات (Normalization أو Standardization) من غير تعديل كود الـ Pipeline.
- استخدم Observer لبناء نظام مراقبة يستجيب لنهاية كل Epoch: Observer لتسجيل الـ Log، وObserver تاني للإيقاف المبكر.
- استخدم Singleton لكلاس إعدادات التجربة (ExperimentConfig) بحيث كل أجزاء الـ Pipeline بتستخدم نفس الإعدادات المتسقة.
- استخدم Dependency Injection لحقن مصدر البيانات وخدمة تسجيل المقاييس، بحيث تقدر تستبدلهم بسهولة بين بيئة الاختبار والإنتاج.

تحديات شائعة عند التطبيق

- الخلط بين Factory وStrategy: الاتنين بيستخدموا واجهة مشتركة، لكن Factory بيركز على إنشاء الكائن المناسب، بينما Strategy بيركز على تبديل السلوك المستخدم بعد الإنشاء. من المفيد تسأل نفسك: هل المشكلة في إزاي أنشئ الكائن (Factory)، ولا في إزاي أستخدمه (Strategy)؟
- المبالغة في استخدام الأنماط الخمسة كلها في مشروع بسيط جدًا: مش كل مشروع محتاج كل الأنماط دي، والاستخدام المفرط بيزود التعقيد من غير فايدة حقيقية. طبق النمط بس لما المشكلة اللي بيحلها موجودة فعلًا.
- نسيان اختبار الـ Observers بشكل منفصل عن بعض: بما إن الـ Observers مفروض يكونوا مستقلين، لازم تتأكد إن الاختبارات بتغطي كل Observer لوحده، مش بس السيناريو الكامل مع كل الـ Observers مع بعض.

إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي

الأنماط الخمسة دي مش مجرد تمارين أكاديمية، هي أدوات يومية في أي نظام ML حقيقي: Factory بيظهر في اختيار النموذج المناسب من الإعدادات، Singleton في إدارة الاتصالات المشتركة، Strategy في تبديل خوارزميات المعالجة والتقييم، Observer في أنظمة الـ Callbacks أثناء التدريب، وDependency Injection في فصل منطق الأعمال عن التفاصيل التقنية القابلة للتغيير.

في الشركات الكبرى، منصات الـ ML الداخلية (Internal ML Platforms) مبنية بالكامل تقريبًا على تركيبة من الأنماط دي، وده اللي بيسمح لفرق النمذجة المختلفة يستخدموا نفس البنية التحتية بمرونة عالية من غير الحاجة لفهم كل التفاصيل الداخلية للمنصة.

ملخص الفصل الثالث وربطه بالفصل القادم

في الفصل ده اتعلمت خمس أنماط تصميم أساسية: Factory (لمركزية إنشاء الكائنات)، Singleton (لضمان نسخة واحدة من مورد مشترك)، Strategy (لتبديل السلوك وقت التشغيل)، Observer (لإشعار عدة أطراف بحدث معين)، وDependency Injection

(لفصل الكلاس عن تفاصيل اعتمادياته). كل نمط من دول شفت إزاي هو تطبيق عملي مباشر لمبدأ أو أكتر من مبادئ SOLID اللي اتعلمناها في الفصل السابق.

الفصل الجاي (Clean Architecture) هياخد كل الأفكار دي لمستوى أعلى: مش بس تصميم كلاس واحد أو علاقة بين كذا كلاس، وإنما تنظيم النظام كله في طبقات (Layers) واضحة، بحيث قاعدة الاعتماد (Dependency Rule) - اللي هي في الأساس امتداد لمبدأ Dependency Inversion ونمط Dependency Injection اللي اتعلمناهم هنا - بتحكم إزاي الطبقات دي بتتفاعل مع بعض عبر المشروع كله.